The Journal of Open Source Software

# MRRpy: A modular Python model for distributed rainfall–runoff generation and flood routing

**Saurav Bhattarai** [1¶], **Nawa Raj Pradhan** [2], **and Rocky Talchabhadel** [1]

**1** Department of Civil and Environmental Engineering, Jackson State University, Jackson, MS, USA **2** Coastal and Hydraulics Laboratory, U.S. Army Engineer Research and Development Center, Vicksburg, MS, USA ¶ Corresponding author

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:** Open Journals

**Reviewers:**

- @openjournals

**Submitted:** 01 January 1970
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

## Summary

MRRpy is an open-source, modular Python hydrological model for distributed rainfall–runoff simulation and channel flood routing, built to be changed and extended. From a digital elevation model (DEM), it delineates a D8 drainage network, builds a precipitation field (uniform, gauged, or satellite), generates runoff (runoff coefficient, Curve Number, or process-based Green–Ampt infiltration-excess, variable-source-area saturation-excess, and impervious mechanisms), and routes it (kinematic wave, Muskingum–Cunge, or a semi-implicit diffusive wave solved exactly on the D8 tree).

Each stage hands the next a single field, so any rainfall source feeds any runoff method and any runoff method drives any routing scheme; a new method in one stage works at once with every method in the others, and an automated test matrix checks every runoff–routing combination for mass conservation. One configuration drives the model from a Python API, a command-line wizard, a Jupyter form, or a QGIS plugin, and upstream inflow hydrographs, virtual gauges, and per-cell depth and velocity maps are available on request. With Google Earth Engine supplying rainfall, soil, and terrain data, an ungauged basin can be set up and analysed quickly. MRRpy installs from PyPI and is documented at https://mrrpy.readthedocs.io.

## Statement of need

Isolating one modeling decision in a flood study is harder than it should be. Comparing two runoff methods with routing held fixed, two routing schemes with runoff held fixed, or gauge against satellite rainfall with both held fixed usually means joining separate research codes or rebuilding the runoff-to-routing-to-diagnostics plumbing by hand, and the inputs, boundary handling, mass accounting, and outputs are easily made inconsistent. The same barrier faces method developers: testing a new infiltration model needs a router, and testing a new router needs runoff. MRRpy factors precipitation, runoff generation, and routing into independent stages behind one configuration, so a comparison is a change of setting, and a new method inherits the rest of the model, its inputs, outputs, and tests. The same structure is meant to grow: further processes, such as groundwater recharge or sediment transport, can be added as new components that reuse the existing terrain, forcing, and routing, and the authors are developing such extensions.

Many runs are as easy as one: a sweep or full factorial over rainfall source, runoff method, routing scheme, and channel geometry is a loop over configuration values, with one comparable ledger row per run and optional GPU acceleration.

MRRpy serves hydrologists running quick analyses, method comparisons, or calibration sweeps; developers testing a new component against established ones; researchers routing a gauge record, reservoir release, or another model's discharge through its boundary-inflow interface;

DRAFT

and students learning each method from readable code and an interactive textbook (https://sauravbhattarai19.github.io/MRRpy/).

## State of the field

`pysheds` (Bartos, 2020) and `pyflwdir` (Eilander & Boisgontier, 2021) are strong Python libraries for D8 terrain analysis but stop at flow directions and delineation; `MRRpy` uses either and continues through runoff, routing, and diagnostics. Landlab (Barnhart et al., 2020) is a general Earth-surface-dynamics framework from whose components a comparable model could be assembled, with more effort. GSSHA (Downer & Ogden, 2004) is a mature, physically based distributed hydrologic model under continued development (Pradhan et al., 2019; Pradhan & Floyd, 2024), and LISFLOOD-FP (Almeida et al., 2012; Bates et al., 2010) solves two-dimensional floodplain flow; neither is designed for adding and swapping runoff or routing methods from Python. We are not aware of an open tool that lets a user combine any rainfall source, runoff method, and routing scheme on the same grid, add a new method that immediately works with all the others, and check every combination for mass conservation. `MRRpy` is built for exactly that; for detailed two-dimensional floodplain hydraulics, a dedicated inundation model remains the right tool.

## Software design

Figure 1 shows the pipeline. One configuration object drives the same pipeline from every interface. A single parameter catalogue generates the wizard, the notebook form, and the manual's settings tables.

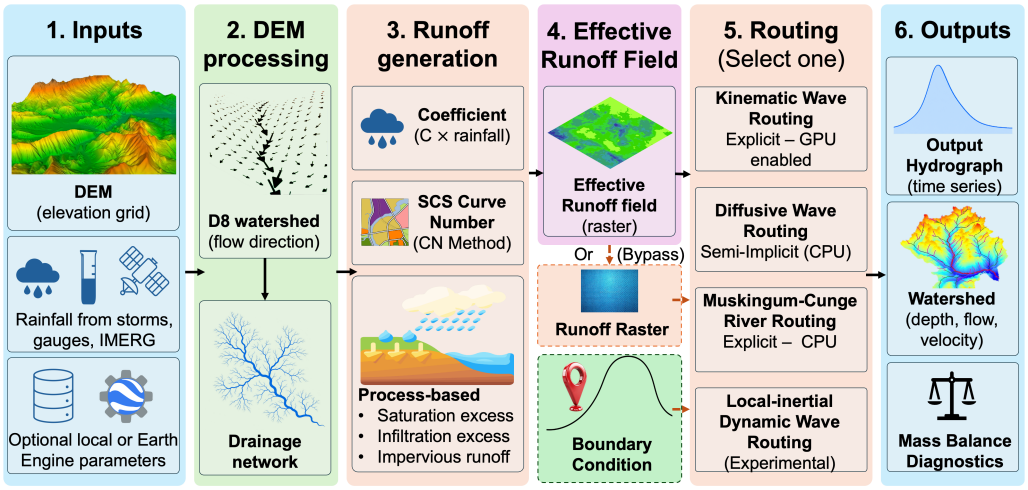



**Figure 1:** MRRpy's processing chain, from terrain and precipitation inputs through D8 delineation and selectable runoff-generation and routing methods to an outlet hydrograph, depth/flow/velocity fields, and a mass-balance diagnostic.

**Terrain and parameters.** DEM processing conditions the terrain (by default least-cost carving (Yamazaki et al., 2012) with `pyflwdir` (Wang & Liu, 2006); `pysheds` (O'Callaghan & Mark, 1984) with filling remains available), computes D8 flow directions, and delineates the catchment above an outlet or keeps the whole DEM; a missing DEM is downloaded from Earth Engine. Soil and land-cover parameters come from local inputs or Earth Engine (Gorelick et al., 2017), including ESA WorldCover (Zanaga et al., 2022), SoilGrids (Poggio et al., 2021), and HiHydroSoil (Simons et al., 2020).

DRAFT

**Precipitation** is its own stage, returning a rain-rate field per cell at any time behind one interface. Sources are a uniform hyetograph, rain gauges interpolated by Thiessen polygons or inverse-distance weighting, and IMERG satellite rainfall (Huffman et al., 2020) downloaded from Earth Engine and treated as a dense pseudo-gauge network with the same interpolators.

**Runoff generation** is a registry of interchangeable modes behind a two-step interface (compute runoff, then update soil state): direct rainfall, a runoff coefficient, SCS Curve Number (Chow et al., 1988; Jaafar et al., 2019), a prescribed raster, and a process-based mode. The process-based mode is itself a second registry of mechanisms composed per cell without counting rainfall twice; the built-in ones are Green–Ampt infiltration excess (Green & Ampt, 1911; Rawls et al., 1983), saturation excess, and impervious shedding, with soil recovery between storms following EPA SWMM 5 (Rossman & Huber, 2016). Saturation excess uses the variable-source-area theory of Pradhan & Ogden (2010), which needs only an initial soil-moisture deficit, here from the satellite-based SERVES method (Pradhan, 2019); developed earlier as a stand-alone theory, it is applied here for distributed simulation alongside standard Green–Ampt. A new method is a class registered with a decorator.

**Routing** schemes are likewise registered. Kinematic wave, variable-parameter Muskingum–Cunge (Cunge, 1969; Ponce & Yevjevich, 1978), and a semi-implicit diffusive wave are supported; an explicit diffusive wave is deprecated and a local-inertial scheme (Almeida et al., 2012) is experimental. Channel cells carry a sub-grid channel (Neal et al., 2012) sized to a bankfull discharge that is user-given or estimated globally (Andreadis et al., 2013; Linke et al., 2019). The semi-implicit scheme applies the classical direct elimination for tree-structured systems to the whole D8 grid, giving an exact implicit solve in linear time without the explicit Courant limit, using Numba (Lam et al., 2015) or SciPy (Virtanen et al., 2020). Kinematic and explicit diffusive routing, runoff, and precipitation can run on a CuPy GPU backend. A discharge hydrograph can be injected at any channel point, with or without rainfall.

**Outputs** are common to every combination: an outlet hydrograph, one ledger row per run (rainfall, boundary inflow, outflow, storage, closure error), and optionally virtual gauges, a per-cell field archive, and runoff split by mechanism, with one-call plotting, animation, and map export.

**Testing.** Because every method meets the same interface, the model is tested as a whole rather than one method at a time. For every change to the main branch, continuous integration (Python 3.9–3.12) runs each supported routing scheme with each runoff method and every subset of the physical mechanisms, with soil recovery on and off, over two storms separated by a dry spell; each run must close its mass balance to $10^{-6}$ and never produce more runoff than rain, and GPU results are compared with CPU where available. A new component added to this matrix is checked against every existing one. Analytic benchmarks and a verification script for the semi-implicit solve (against dense and sparse solvers and the Hayami solution) cover the numerics.

## Research impact statement

MRRpy is designed as a foundation for research rather than a single application, and its impact rests on five capabilities.

First, it makes ungauged basins workable. Terrain, satellite rainfall, soil, and land cover can all come from Earth Engine, so a first model needs no local data, and running it across the available satellite products and a range of runoff and routing assumptions gives a bounded estimate quickly enough to support early warning. Where gauge records, field surveys, or local rasters exist, they replace the global inputs through the same configuration, so a model can start ungauged and improve as data arrive.

Second, it supports analysis at every scale of effort: a single event, continuous multi-year simulation with soil recovery between storms, or hundreds of scenarios in one sweep, every

DRAFT

run on one comparable ledger. Rainfall and upstream inflow can drive the same run, and the saved depth, velocity, and discharge fields show the flood at any location.

Third, it rests on physically based processes and established numerical schemes with verified mass conservation, so its results can be explained. Initial simulations of observed Bagmati River floods at Khokana, Nepal, driven by government rain gauges and automatic weather stations installed by SmartPhones4Water, agree encouragingly with observed discharge and, for the September 2024 flood, with the timing and shape of observed water levels at most of ten Kathmandu Valley stations.

Fourth, it serves teaching and learning. The textbook, the worked examples, and readable code let students learn each method, apply it to their own basin, and estimate floods, and its four interfaces open it to programmers and GIS practitioners alike.

Finally, it is a beginning. Follow-up papers will present new runoff-generation physics built on the mechanism registry and an application to ungauged basins validated against observed floods. Further processes, a stable local-inertial scheme, and a two-dimensional shallow-water model that zooms into selected areas to map inundation from the routed flows are in development.

## AI usage disclosure

Anthropic's Claude Code CLI (Claude Sonnet 4.6, Opus 4.6, Opus 4.8, Sonnet 5, Fable 5, and Opus 5.5) was used for code implementation, refactoring, test scaffolding, documentation, and drafting and revising this manuscript. All AI-assisted code and text were reviewed, tested, and edited by the human authors, who are responsible for the correctness of the software, the validity of the reported results, and the content of this paper.

## Acknowledgements

`MRRpy` is built on the open-source scientific-Python and geospatial ecosystem, including NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), `rasterio`, GeoPandas, `pysheds`, and `pyflwdir`, and we thank their developers and maintainers.

This research was supported in part by an appointment to the Department of Defense (DOD) Research Participation Program administered by the Oak Ridge Institute for Science and Education (ORISE) through an interagency agreement between the U.S. Department of Energy (DOE) and the DOD. ORISE is managed by ORAU under DOE contract number DE-SC0014664. All opinions expressed in this paper are the authors' and do not necessarily reflect the policies and views of DOD, DOE, or ORAU/ORISE.

This research was also supported by the U.S. Army Engineer Research and Development Center, Coastal and Hydraulics Laboratory (ERDC–CHL), through its Earth System Hydrology research initiative, with technical support provided under the ERDC–CHL professional services contract with Taylor–Woolpert, LLC.

`MRRpy`, an open-source hydrologic modeling software package, was developed through research collaboration between the U.S. Army Engineer Research and Development Center, Coastal and Hydraulics Laboratory (ERDC–CHL), and the Hydrology Laboratory at Jackson State University (JSU).

## References

Almeida, G. A. M. de, Bates, P., Freer, J. E., & Souvignet, M. (2012). Improving the stability of a simple formulation of the shallow water equations for 2-D flood modeling. *Water Resources Research*, *48*(5), W05528. https://doi.org/10.1029/2011WR011570

DRAFT

Andreadis, K. M., Schumann, G. J.-P., & Pavelsky, T. (2013). A simple global river bankfull width and depth database. *Water Resources Research*, *49*(10), 7164–7168. https://doi.org/10.1002/wrcr.20440

Barnhart, K. R., Hutton, E. W. H., Tucker, G. E., Gasparini, N. M., Istanbulluoglu, E., Hobley, D. E. J., Lyons, N. J., Mouchene, M., Nudurupati, S. S., Adams, J. M., & Bandaragoda, C. (2020). Short communication: Landlab v2.0: A software package for earth surface dynamics. *Earth Surface Dynamics*, *8*(2), 379–397. https://doi.org/10.5194/esurf-8-379-2020

Bartos, M. (2020). *Pysheds: Simple and fast watershed delineation in python*. https://doi.org/10.5281/zenodo.3822494

Bates, P. D., Horritt, M. S., & Fewtrell, T. J. (2010). A simple inertial formulation of the shallow water equations for efficient two-dimensional flood inundation modelling. *Journal of Hydrology*, *387*(1–2), 33–45. https://doi.org/10.1016/j.jhydrol.2010.03.027

Chow, V. T., Maidment, D. R., & Mays, L. W. (1988). *Applied hydrology*. McGraw-Hill.

Cunge, J. A. (1969). On the subject of a flood propagation computation method (muskingum method). *Journal of Hydraulic Research*, *7*(2), 205–230. https://doi.org/10.1080/00221686909500264

Downer, C. W., & Ogden, F. L. (2004). GSSHA: Model to simulate diverse stream flow producing processes. *Journal of Hydrologic Engineering*, *9*(3), 161–174. https://doi.org/10.1061/(ASCE)1084-0699(2004)9:3(161)

Eilander, D., & Boisgontier, H. (2021). *Pyflwdir: Fast methods to work with hydro- and topography data in pure python*. https://github.com/Deltares/pyflwdir

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, *202*, 18–27. https://doi.org/10.1016/j.rse.2017.06.031

Green, W. H., & Ampt, G. A. (1911). Studies on soil phyics: 1. The flow of air and water through soils. *The Journal of Agricultural Science*, *4*(1), 1–24. https://doi.org/10.1017/S0021859600001441

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Huffman, G. J., Bolvin, D. T., Braithwaite, D., Hsu, K.-L., Joyce, R., Kidd, C., Nelkin, E. J., Sorooshian, S., Tan, J., & Xie, P. (2020). *NASA global precipitation measurement (GPM) integrated multi-satellitE retrievals for GPM (IMERG)* [Algorithm Theoretical Basis Document (ATBD), version 06]. NASA.

Jaafar, H. H., Ahmad, F. A., & El Beyrouthy, N. (2019). GCN250, new global gridded curve numbers for hydrologic modeling and design. *Scientific Data*, *6*(1), 145. https://doi.org/10.1038/s41597-019-0155-x

Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT compiler. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6. https://doi.org/10.1145/2833157.2833162

Linke, S., Lehner, B., Ouellet Dallaire, C., Ariwi, J., Grill, G., Anand, M., Beames, P., Burchard-Levine, V., Maxwell, S., Moidu, H., Tan, F., & Thieme, M. (2019). Global hydro-environmental sub-basin and river reach characteristics at high spatial resolution. *Scientific Data*, *6*, 283. https://doi.org/10.1038/s41597-019-0300-6

Neal, J., Schumann, G., & Bates, P. (2012). A subgrid channel model for simulating river hydraulics and floodplain inundation over large and data sparse areas. *Water Resources*

DRAFT

*Research*, *48*(11). https://doi.org/10.1029/2012WR012514

O'Callaghan, J. F., & Mark, D. M. (1984). The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics, and Image Processing*, *28*(3), 323–344. https://doi.org/10.1016/S0734-189X(84)80011-0

Poggio, L., Sousa, L. M. de, Batjes, N. H., Heuvelink, G. B. M., Kempen, B., Ribeiro, E., & Rossiter, D. (2021). SoilGrids 2.0: Producing soil information for the globe with quantified spatial uncertainty. *SOIL*, *7*(1), 217–240. https://doi.org/10.5194/soil-7-217-2021

Ponce, V. M., & Yevjevich, V. (1978). Muskingum-cunge method with variable parameters. *Journal of the Hydraulics Division*, *104*(12), 1663–1667. https://doi.org/10.1061/JYCEAJ.0005186

Pradhan, N. R. (2019). Estimating growing-season root zone soil moisture from vegetation index-based evapotranspiration fraction and soil properties in the Northwest Mountain region, USA. *Hydrological Sciences Journal*, *64*(7), 771–788. https://doi.org/10.1080/02626667.2019.1593417

Pradhan, N. R., Downer, C. W., & Marchenko, S. (2019). Catchment hydrological modeling with soil thermal dynamics during seasonal freeze-thaw cycles. *Water*, *11*(1), 116. https://doi.org/10.3390/w11010116

Pradhan, N. R., & Floyd, I. (2024). Examining the effect of moisture thresholds on post-fire water-repellent soil: A large-scale modelling approach applied to the Upper Arroyo Seco watershed, California, USA. *International Journal of Wildland Fire*, *33*(4), WF22083. https://doi.org/10.1071/WF22083

Pradhan, N. R., & Ogden, F. L. (2010). Development of a one-parameter variable source area runoff model for ungauged basins. *Advances in Water Resources*, *33*(5), 572–584. https://doi.org/10.1016/j.advwatres.2010.03.002

Rawls, W. J., Brakensiek, D. L., & Miller, N. (1983). Green-ampt infiltration parameters from soils data. *Journal of Hydraulic Engineering*, *109*(1), 62–70. https://doi.org/10.1061/(ASCE)0733-9429(1983)109:1(62)

Rossman, L. A., & Huber, W. C. (2016). *Storm water management model reference manual volume I – hydrology (revised)* (No. EPA/600/R-15/162A). U.S. Environmental Protection Agency.

Simons, G. W. H., Koster, R., & Droogers, P. (2020). *HiHydroSoil v2.0 – a high resolution soil map of global hydraulic properties* (FutureWater Report No. 213). FutureWater. https://www.futurewater.nl/wp-content/uploads/2020/10/HiHydroSoil-v2.0-High-Resolution-Soil-Maps-of-Global-Hydraulic-Properties.pdf

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wang, L., & Liu, H. (2006). An efficient method for identifying and filling surface depressions in digital elevation models for hydrologic analysis and modelling. *International Journal of Geographical Information Science*, *20*(2), 193–213. https://doi.org/10.1080/13658810500433453

Yamazaki, D., Baugh, C. A., Bates, P. D., Kanae, S., Alsdorf, D. E., & Oki, T. (2012). Adjustment of a spaceborne DEM for use in floodplain hydrodynamic modeling. *Journal of Hydrology*, *436–437*, 81–91. https://doi.org/10.1016/j.jhydrol.2012.02.045

Zanaga, D., Van De Kerchove, R., Daems, D., De Keersmaecker, W., Brockmann, C., Kirches,

DRAFT

G., Wevers, J., Cartus, O., Santoro, M., Fritz, S., Lesiv, M., Herold, M., Tsendbazar, N.-E., Xu, P., Ramoino, F., & Arino, O. (2022). *ESA WorldCover 10 m 2021 v200*. https://doi.org/10.5281/zenodo.7254221

DRAFT